AUSTCAIC — AI & ML WING

Hands-on Weekends: AI & ML Starter Culture

Class1: Python for Machine Learning

Session Date: 18 July 2026

Time: 9 PM

OVERVIEW

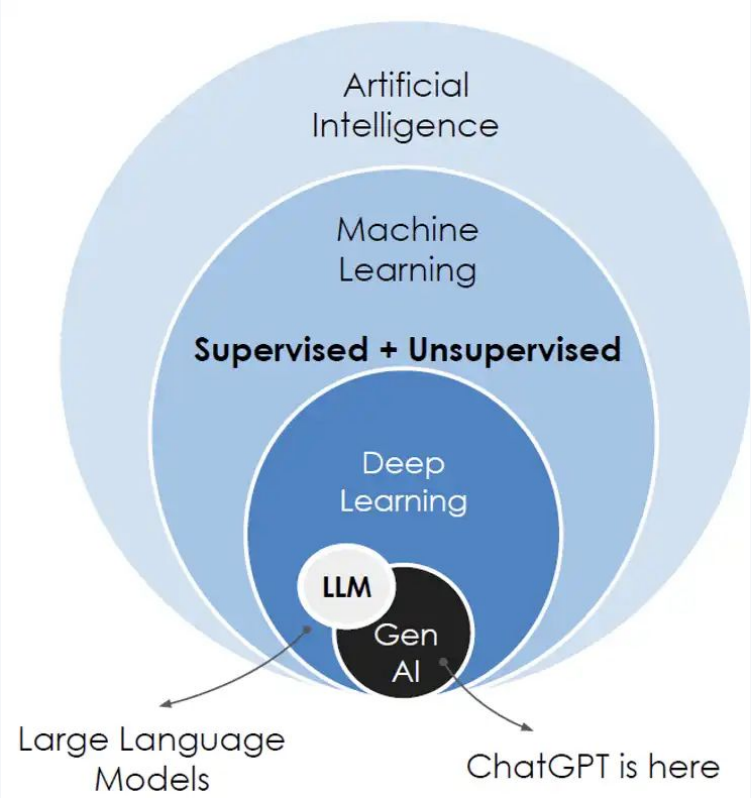
What We'll Cover Today

Nine building blocks, one working calculator by the end.

1. Introduction to AI & Machine Learning
2. Introduction to Python
3. Input and Output
4. Variables and Data Types
5. Operators
6. Data Structures — Lists, Tuples, Dicts, Sets
7. Conditionals
8. Loops
9. Functions
10. **Task — Build a Simple Calculator**

What is AI & Machine Learning?

- Artificial Intelligence (AI): machines performing tasks that normally need human intelligence.
- Machine Learning (ML): a subset of AI — systems that learn patterns from data instead of following fixed rules.
- Deep Learning (DL): a subset of ML using multi-layered neural networks.
- Today's focus: Python — the most famous programming language for AI and ML.



Introduction to Python

- Created by Guido van Rossum, first released in 1991.
- Named after the comedy show "Monty Python's Flying Circus"
- Philosophy: Easy to read, easy to learn .
- Why Python for ML: huge ecosystem — NumPy, Pandas, PyTorch, scikit-learn etc.
- Interpreted, dynamically typed, beginner-friendly syntax.





Variables & Data Types

- A variable is a name that stores a value in memory.
- **Core built-in types:**
  - int, float, str, bool
- No need to declare a type — Python infers it automatically.

```
name = "Rahim"      # str
age = 24             # int
gpa = 3.5            # float
is_student = True    # bool

print(type(age))
# <class 'int'>
```

Input and Output

- print() displays output to the screen.
- input() reads text typed by the user.
- input() always returns a string — convert it (int(), float()) when needed.
- f-strings make formatted output easy.

```
name = input("Your name: ")
age = int(input("Your age: "))

print(f"Hi {name}, "
      f"next year you'll be "
      f"{age + 1}")
```

Operators

Arithmetic	`+ - * / // % **`	Math operations, floor-div & power
Comparison	`== != > < >= <=`	Returns True / False
Logical	`and or not`	Combine boolean conditions
Assignment	`= += -= *= /=`	Assign & update in one step

Conditionals

- **What is a conditional?** A piece of code that checks something and decides what to do next, based on whether it's true or false.

- **Why we use it:** so that our program can react differently depending on the situation, instead of always doing the same thing.

- **if:** runs a block of code only when a condition is true.
- **elif:** checks another condition, if the first one was false.
- **else:** runs when none of the above conditions were true.

- **Indentation matters:** Python uses spacing (not { }) to show which lines belong to which condition.

```
marks = 78

if marks >= 80:
    grade = 'A'
elif marks >= 60:
    grade = 'B'
else:
    grade = 'C'

print(grade)  # B
```

Lists & Tuples

List

- Ordered, mutable collection
- Written with [ ]
- Allows duplicates

```
fruits = ['apple', 'kiwi']
fruits.append('mango')
print(fruits[0])  # apple
```

Tuple

- Ordered, immutable collection
- Written with ( )
- Faster than lists; used for fixed data

```
point = (4, 7)
x, y = point
# point[0] = 9  -> Error!
```

Dictionaries & Sets

Dictionary

- Key–value pairs
- Written with { key: value }
- Fast lookup by key

```
student = {'name': 'Rahim',
           'dept': 'CSE'}
print(student['dept'])
```

Set

- Unordered, unique elements
- Written with { }
- Great for removing duplicates

```
nums = {1, 2, 2, 3}
print(nums)  # {1, 2, 3}
```

Slicing in Python

- Slicing grabs a portion of a sequence using [start:stop:step].
- start = where to begin (included)
- stop = where to end (excluded)
- step = how many to skip each time (optional)
- Negative numbers count from the end (-1 = last item).
- **Same rules apply to strings, lists, and tuples**.

```
word = "python"

print(word[0:3])    # 'pyt'
print(word[:3])     # 'pyt'
print(word[2:])     # 'thon'
print(word[-1])     # 'n'
print(word[::-1])   # 'nohtyp'

nums = [10, 20, 30, 40]
print(nums[1:3])    # [20, 30]
```

Loops

- **A loop repeats a block of code** multiple times, instead of writing it over and over.
- **Why we use loops:** to avoid writing the same line again and again, and to go through items one by one automatically.
- **for loop:** iterates over a sequence (list, range, string) — best when you know what you're looping through.
- **while loop:** repeats as long as a condition stays True — best when you don't know exactly how many times, you'll iterate.

```
for i in range(3):
    print(i)    # 0 1 2

count = 0
while count < 3:
    print(count)
    count += 1
```

Functions

- **What is a function?** A named block of code that performs a specific task, which you can reuse whenever you need it.
- **Why we use it:** so that we don't have to write the same code again and again — we just call the function by name.
- **def:** the keyword used to create a function.
- **Parameters:** values you can pass into a function to work with.
- **return:** sends a result back out of the function.
- **Keep functions small and focused on one task** — easier to read, test, and reuse.

```
def add(a, b):
    return a + b

def greet(name="there"):
    print(f"Hello, {name}!")

print(add(3, 5))   # 8
greet("Rahim")
```

!

Task

Build a Simple Calculator

Follow the steps in order — try it on your own before asking for help.

1. Ask the user to enter the first number.
2. Ask the user to choose an operation: +, -, *, /
3. Ask the user to enter the second number.
4. Use if / elif / else to check which operation was chosen.
5. Store the calculated result in a variable.
6. Print the result using an f-string.
7. **Bonus: wrap everything in a while loop so the user can keep calculating until they type 'quit'.**